



NATIONAL UNIVERSITY OF MODERN LANGUAGES

SOFTWARE QUALITY ENGINEERING LAB REPORT – 8

NAME: Abdul Rafay

ROLL NO: FL23791

PROGRAM: BSSE (6th Semester)

COURSE: SOFTWARE QUALITY ENGINEERING

SUBMITTED TO: Mam Rubab

DATE: 9 May 2026

DAY: Saturday

Contents

1. Introduction.....	3
2. Test Objectives.....	3
3. Scope of Testing.....	3
4. Testing Approach	3
5. Test Levels	4
6. Roles and Responsibilities	4
7. Test Environment	4
8. Entry and Exit Criteria	4
BLACK BOX TEST CASES.....	5
Module 1: User Registration.....	5
Module 2: User Login.....	5
Module 3: Doctor Search.....	6
Module 4: Appointment Booking	6
Module 5: Doctor Appointment Management	7
Module 6: Admin Functions	7
WHITE BOX TEST CASES	8
Module 1: User Registration Logic.....	8
Module 2: User Login Logic.....	8
Module 3: Appointment Booking Logic	9
Module 4: Appointment Approval Logic.....	9
Module 5: Admin Doctor Management Logic	10
9. Defect Reporting.....	11
10. Conclusion	11

Doctor Appointment Booking System Test Plan

1. Introduction

This Test Plan has been prepared for the Doctor Appointment Booking System. The purpose of this document is to describe how the testing of the system will be carried out, what will be tested, who will do the testing, and what tools and techniques will be used.

Testing is a critical activity in software development. According to Pressman (2014), testing is the process of executing a program with the intent of finding errors. This plan ensures that all parts of the system are properly tested before the final release.

2. Test Objectives

The objectives of this test plan are:

- To verify that all functional requirements of the system work correctly
- To validate that the system meets the needs of the users
- To identify and report defects before the system is released
- To ensure the system meets non-functional requirements such as performance, security, and usability
- To confirm that fixed defects do not break other parts of the system

3. Scope of Testing

The following modules and features are included in the scope of testing:

- User Registration
- User Login and Authentication
- Doctor Search
- Appointment Booking
- Appointment Status View by Patient
- Appointment Approval and Rejection by Doctor
- Doctor Management by Admin
- User Management by Admin

4. Testing Approach

Two main testing techniques will be used in this project:

Black Box Testing — Tests the system from the user's point of view without looking at the internal code. The tester only checks inputs and expected outputs.

White Box Testing — Tests the internal logic and structure of the code. The tester examines the code paths, conditions, and branches to make sure every part of the code is covered.

5. Test Levels

- Unit Testing — Each module is tested individually
- Integration Testing — Modules are tested together to check their interaction
- System Testing — The complete system is tested as a whole
- User Acceptance Testing — Real users test the system to confirm it meets their needs

6. Roles and Responsibilities

Role	Responsibility
Test Lead	Prepare test plan, assign test cases, monitor progress
Testers	Write and execute test cases, report defects
Developers	Fix reported defects, support unit testing
Project Manager	Review and approve the test plan

7. Test Environment

- Operating System: Windows 10 or above
- Browser: Google Chrome, Microsoft Edge
- Frontend: React.js
- Backend: Node.js, Express.js
- Database: MongoDB
- Tools: Postman for API testing, VS Code for code inspection

8. Entry and Exit Criteria

Entry Criteria — Testing begins when:

- All modules have been developed and reviewed
- Test cases have been written and approved
- Test environment is set up and ready

Exit Criteria — Testing ends when:

- At least 90% of test cases have passed
- All critical and major defects have been resolved
- No new critical defects are found for two consecutive test cycles

BLACK BOX TEST CASES

Black box testing focuses on inputs and expected outputs without looking at the internal code. It tests the system from the user's perspective.

Module 1: User Registration

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-01	Register with valid details	Name: Ali, Email: ali@gmail.com, Password: Ali@123	Account created successfully, user redirected to login	
BB-02	Register with already used email	Email already in database	Error message: "Email already exists"	
BB-03	Register with empty name field	Name field left blank	Error message: "Name is required"	
BB-04	Register with invalid email format	Email: alimail.com	Error message: "Enter a valid email address"	
BB-05	Register with short password	Password: 123	Error message: "Password must be at least 6 characters"	

Module 2: User Login

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-06	Login with correct credentials	Email: ali@gmail.com, Password: Ali@123	Login successful, user taken to dashboard	
BB-07	Login with wrong password	Email: ali@gmail.com, Password: wrongpass	Error message: "Invalid credentials"	
BB-08	Login with unregistered email	Email: unknown@gmail.com	Error message: "User not found"	

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-09	Login with empty email field	Email field left blank	Error message: "Email is required"	
BB-10	Login with empty password field	Password field left blank	Error message: "Password is required"	

Module 3: Doctor Search

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-11	Search with valid doctor name	Search: "Dr. Ahmed"	Doctor profile displayed	
BB-12	Search with valid specialization	Search: "Cardiologist"	List of cardiologist doctors displayed	
BB-13	Search with name that does not exist	Search: "Dr. XYZ"	Message: "No doctor found"	
BB-14	Search with empty input	Search field left blank	All available doctors displayed	

Module 4: Appointment Booking

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-15	Book appointment with valid details	Doctor selected, valid time slot chosen	Appointment booked successfully, confirmation shown	
BB-16	Book appointment without selecting time slot	Doctor selected but no time slot	Error message: "Please select a time slot"	
BB-17	Book appointment on already taken slot	Slot already booked by another patient	Error message: "This slot is not available"	

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-18	View appointment status after booking	Logged in as patient	Appointment status shown as Pending	

Module 5: Doctor Appointment Management

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-19	Doctor approves an appointment	Doctor clicks Approve	Appointment status changes to Approved	
BB-20	Doctor rejects an appointment	Doctor clicks Reject	Appointment status changes to Rejected	
BB-21	Doctor views all pending appointments	Doctor logs in	All pending appointments listed correctly	

Module 6: Admin Functions

Test Case ID	Test Description	Input	Expected Output	Pass / Fail
BB-22	Admin adds a new doctor	Valid doctor details entered	Doctor added and visible in the system	
BB-23	Admin adds doctor with missing fields	Required field left blank	Error message: "All fields are required"	
BB-24	Admin deletes a doctor	Admin selects a doctor and clicks Delete	Doctor removed from the system	
BB-25	Admin views all registered patients	Admin goes to patient management	Full list of registered patients displayed	

WHITE BOX TEST CASES

White box testing examines the internal logic and code structure. It ensures every code path, condition, and branch is tested properly. According to Pressman (2014), white box testing uses the structure of the program to derive test cases.

Module 1: User Registration Logic

Code Logic Being Tested:

The registration function checks if the email already exists in the database. If it does, it returns an error. If not, it hashes the password and saves the user.

Conditions to test:

- Email exists in database → return error
- Email does not exist → hash password → save user → return success

Test Case ID	Test Description	Condition Tested	Expected Result	Pass / Fail
WB-01	Email already exists in database	if (existingUser) → true	Function returns "Email already exists" error	
WB-02	Email does not exist	if (existingUser) → false	Password is hashed and user is saved to database	
WB-03	Password hashing executed	After email check passes	Password stored as hashed value, not plain text	
WB-04	All required fields validated	Any field is empty	Validation catches empty field before database query	

Module 2: User Login Logic

Code Logic Being Tested:

The login function finds the user by email. If not found, it returns an error. If found, it compares the entered password with the stored hashed password. If they match, a session token is generated.

Conditions to test:

- User not found → return error
- User found, password wrong → return error
- User found, password correct → generate token → return success

Test Case ID	Test Description	Condition Tested	Expected Result	Pass / Fail
WB-05	User not found by email	if (!user) → true	Function returns "User not found" error	
WB-06	User found but password does not match	if (passwordMatch) → false	Function returns "Invalid credentials" error	
WB-07	User found and password matches	if (passwordMatch) → true	Token is generated and returned to client	
WB-08	Token generation branch executed	After successful password match	Token is created with correct user ID and expiry	

Module 3: Appointment Booking Logic

Code Logic Being Tested:

The booking function checks if the selected time slot is already taken. If it is, it returns an error. If not, it creates a new appointment record with a status of Pending.

Conditions to test:

- Slot already booked → return error
- Slot is available → create appointment → return success

Test Case ID	Test Description	Condition Tested	Expected Result	Pass / Fail
WB-09	Selected slot is already booked	if (existingAppointment) → true	Function returns "Slot not available" error	
WB-10	Selected slot is available	if (existingAppointment) → false	New appointment is created with status Pending	
WB-11	Appointment saved to database	After availability check passes	Appointment record exists in MongoDB with correct details	
WB-12	Unauthenticated user tries to book	Auth middleware check fails	Request is blocked, error message returned	

Module 4: Appointment Approval Logic

Code Logic Being Tested:

The doctor approval function finds the appointment by ID and checks if it belongs to the logged-in doctor. If yes, it updates the status to Approved or Rejected based on the doctor's action.

Conditions to test:

- Appointment not found → return error
- Appointment found but belongs to a different doctor → return error
- Appointment found and belongs to correct doctor → update status

Test Case ID	Test Description	Condition Tested	Expected Result	Pass / Fail
WB-13	Appointment ID not found	if (!appointment) → true	Function returns "Appointment not found" error	
WB-14	Appointment belongs to a different doctor	if (appointment.doctorId !== loggedInDoctor) → true	Access denied error returned	
WB-15	Doctor approves valid appointment	Status update branch for Approved	Appointment status updated to Approved in database	
WB-16	Doctor rejects valid appointment	Status update branch for Rejected	Appointment status updated to Rejected in database	

Module 5: Admin Doctor Management Logic

Code Logic Being Tested:

The admin function for adding a doctor checks if all required fields are provided. If any field is missing, it returns a validation error. If all fields are present, it saves the new doctor to the database.

Conditions to test:

- Required fields missing → return validation error
- All fields present → save doctor → return success

Test Case ID	Test Description	Condition Tested	Expected Result	Pass / Fail
WB-17	Required field is missing	Validation check → fails	Function returns "All fields are required" error	
WB-18	All required fields provided	Validation check → passes	New doctor saved to database successfully	
WB-19	Non-admin user tries to add doctor	Role check middleware → fails	Request blocked, access denied error returned	

Test Case ID	Test Description	Condition Tested	Expected Result	Pass / Fail
WB-20	Delete function removes correct record	Doctor ID matched in database	Correct doctor record deleted, others unaffected	

9. Defect Reporting

Any defect found during testing will be reported using the following format:

Field	Description
Defect ID	Unique ID for the defect
Module	The module where the defect was found
Test Case ID	The test case that found the defect
Description	What went wrong
Severity	Critical, Major, Minor, or Trivial
Status	Open, In Progress, Fixed, Closed

10. Conclusion

This test plan along with the black box and white box test cases provides a complete testing framework for the Doctor Appointment Booking System. Black box testing ensures the system works correctly from the user's point of view, while white box testing ensures the internal logic and code paths are functioning as expected. Together they provide thorough coverage of the system and help deliver a reliable and quality product.